# Lab 9 & 10 Report

## High-Concurrency Ledger Service


**Group 4**

Subhanshu Gupta (B24CS1072)

Anarghya Das (B24CS1096)

Arnav Vivek (B24CS1014)

Neeli Satwik (B24CS1048)


Date: April 3, 2026

# SECTION I: SOFTWARE METRICS

This section quantifies engineering effort, delivery flow, code complexity, and quality risk using a blended view of repository evidence and tracker data. For this distributed financial system, metrics covering sizing (COCOMO/FP), algorithmic complexity (Halstead), agile flow (CFD/throughput/burndown/burnup), and reliability exposure (churn, coverage, defect density) are presented.

## 1a. Intermediate COCOMO (Embedded Mode)

Embedded mode is chosen because the SRS v3.0 specifies ACID-like guarantees, two-phase commit for cross-shard atomicity, and synchronous WAL fsync durability — all of which impose tight correctness and timing constraints typical of high-assurance systems.

### Repository Sizing Inputs

The repository consists of 52 Go files and 306 functions. The complete file-level breakdown is shown below:

| Go File | Functions | LOC |
|---|---|---|
| api/handlers/transaction.go | 3 | 83 |
| api/kafka/producer.go | 3 | 51 |
| api/middleware/auth.go | 1 | 48 |
| api/middleware/rate_limiter.go | 3 | 64 |
| api/middleware/rbac.go | 1 | 26 |
| cmd/api/main.go | 2 | 64 |
| cmd/coordinator/main.go | 2 | 109 |
| cmd/devtoken/main.go | 1 | 25 |
| cmd/fault-proxy/main.go | 5 | 155 |
| cmd/load-monitor/main.go | 2 | 60 |
| cmd/loadgen/main.go | 3 | 220 |
| cmd/shard/main.go | 2 | 103 |
| coordinator/consumer/consumer.go | 16 | 371 |
| coordinator/consumer/kafka_consumer.go | 3 | 67 |
| coordinator/router/router.go | 4 | 81 |
| coordinator/shardmap/shard_map.go | 10 | 204 |
| coordinator/twopc/coordinator.go | 5 | 131 |
| internal/config/config.go | 3 | 130 |
| load-monitor/monitor.go | 10 | 230 |

| Go File | Functions | LOC |
|---|---|---|
| main.go | 0 | 1 |
| messaging/client.go | 9 | 193 |
| shard/failover/election.go | 2 | 135 |
| shard/failover/election_test.go | 2 | 84 |
| shard/failover/heartbeat.go | 5 | 106 |
| shard/failover/heartbeat_test.go | 2 | 57 |
| shard/ledger/ledger.go | 15 | 258 |
| shard/partition/partition.go | 10 | 137 |
| shard/recovery/recovery.go | 2 | 178 |
| shard/replication/client.go | 3 | 80 |
| shard/replication/follower.go | 6 | 92 |
| shard/replication/primary.go | 4 | 136 |
| shard/server/http_handler.go | 20 | 398 |
| shard/server/shard_server.go | 28 | 577 |
| shard/wal/wal.go | 13 | 372 |
| shared/constants/states.go | 0 | 41 |
| shared/models/transaction.go | 0 | 25 |
| shared/models/wal_entry.go | 0 | 61 |
| shared/utils/hash.go | 5 | 48 |
| storage/engine.go | 0 | 35 |
| storage/json_store.go | 9 | 144 |
| tests/integration/concurrency_test.go | 8 | 252 |
| tests/integration/cross_shard_test.go | 8 | 285 |
| tests/integration/replication_test.go | 7 | 207 |
| tests/integration/single_shard_test.go | 7 | 252 |
| tests/unit/config_test.go | 6 | 100 |
| tests/unit/hash_test.go | 6 | 104 |
| tests/unit/ledger_test.go | 13 | 215 |
| tests/unit/partition_test.go | 9 | 135 |
| tests/unit/shard_map_test.go | 7 | 184 |
| tests/unit/storage_test.go | 8 | 209 |
| tests/unit/wal_checkpoint_test.go | 5 | 180 |

| Go File | Functions | LOC |
|---|---|---|
| tests/unit/wal_test.go | 8 | 193 |

### Nominal COCOMO Calculation

Using the lab rule of 35 LOC/function:

```
Base LOC = 306 × 35 = 10,710
Estimated LOC (with 20% overhead) = 10,710 × 1.20 = 12,852
KLOC = 12,852 / 1000 = 12.852 KLOC
```

Constants for Embedded Mode: a = 3.6, b = 1.20, c = 2.5, d = 0.32

```
Nominal Effort = a × (KLOC)^b = 3.6 × (12.852)^1.20 → 77.10 person-months
Nominal Duration = c × (Effort)^d = 2.5 × (77.10)^0.32 → 10.04 months
Average Staff = 77.10 / 10.04 → 7.68 people
```

### Cost Driver Ratings (EAF Calculation)

| Driver | Rating | Multiplier | Evidence-Based Justification |
|---|---|---|---|
| RELY | Very High | 1.40 | Financial correctness and ACID guarantees in SRS |
| DATA | High | 1.08 | State-heavy ledger, WAL, shard maps |
| CPLX | High | 1.15 | 2PC + replication + failover interactions |
| TIME | High | 1.11 | Latency requirements REQ-PERF-002/003 |
| STOR | Nominal+ | 1.06 | WAL persistence and replay retention |
| VIRT | High | 1.10 | Dockerized distributed deployment |
| TURN | High | 1.07 | Frequent sprint iterations and updates |
| ACAP | High | 0.86 | Team experience reflected by broad module ownership |
| AEXP | High | 0.91 | Prior distributed systems coursework context |
| PCAP | High | 0.86 | Consistent code/test throughput in sprints |
| VEXP | Low | 1.10 | Kafka + distributed stack introduced mid-project |
| LEXP | Nominal | 0.95 | Go used throughout project |
| MODP | High | 0.91 | Tracker-driven process discipline |
| TOOL | High | 0.91 | GitHub, Docker, automated test scripts |
| SCED | Very High | 1.10 | Sprint compression/slippage pressure |

```
EAF = product of all 15 multipliers → 1.5427
```

```
Adjusted Effort = Nominal Effort × EAF = 77.10 × 1.5427 → 118.95 person-months
Adjusted Duration = 2.5 × (118.95)^0.32 → 11.54 months
Adjusted Staff = 118.95 / 11.54 → 10.31 people
```

*Interpretation: COCOMO indicates a non-trivial team load for an academic schedule. The EAF uplift is driven primarily by reliability (RELY = 1.4) and complexity (CPLX = 1.15) drivers, consistent with the observed Sprint 3 extension and the concentration of Ongoing items in coordination and replication modules.*

## 1b. Halstead Metrics (WAL Unit)

Analysis unit: shard/wal/wal.go under REQ-DATA-002. The implemented function signatures are used while preserving the same functional intent as the tracker entries (NewWAL/Fsync). Token counts are collected from: Open, Append, ReadFrom, Truncate, and writeEntry.

| Function | Actual Signature |
|---|---|
| Open | func Open(filePath string) (*WAL, error) |
| Append | func (w *WAL) Append(txnID string, opType constants.OperationType, accountID string, amount int64) (uint64, error) |
| writeEntry | func (w *WAL) writeEntry(entry models.WALEntry) error |
| ReadFrom | func (w *WAL) ReadFrom(startLogID uint64) ([]models.WALEntry, error) |
| Truncate | func (w *WAL) Truncate(beforeLogID uint64) error |

Operator/Operand counts extracted from the five functions:

```
n1 (distinct operators) = 20
n2 (distinct operands) = 206
N1 (total operators) = 214
N2 (total operands) = 584
```

### Halstead Metrics Summary

| Metric | Formula | Value |
|---|---|---|
| Program vocabulary (η) | n1 + n2 | 226.00 |
| Program length (N) | N1 + N2 | 798.00 |
| Estimated length (N̂) | $n1 \cdot \log_2(n1) + n2 \cdot \log_2(n2)$ | 1669.86 |
| Volume (V) | $N \cdot \log_2(\eta)$ | 6240.50 |
| Difficulty (D) | $(n1/2) \cdot (N2/n2)$ | 28.35 |
| Effort (E) | D · V | 176,915.23 |

| Metric | Formula | Value |
|---|---|---|
| Time (T) | E / 18 seconds | 9828.62 s |
| Predicted bugs (B) | E^(2/3) / 3000 | 1.050 |

*Interpretation: WAL code carries substantial algorithmic effort (E ≈ 176,915 Halstead units) due to strict durability and truncate/rewrite paths. The predicted bug count of ~1.05 underscores the need for rigorous fault-injection coverage in stateful coordinator-replication interactions.*

## 1c. Function Point Analysis

FPA counts were derived by mapping SRS v3.0 requirements and implementation tracker entries into user-recognizable transactions and logical files. Where DET/FTR granularity is not explicit in the tracker, complexity assignment is marked as an assumption.

| Type | Complexity Split | Weights (L/A/H) | Weighted Count |
|---|---|---|---|
| EI (External Input) | 2 Low, 4 Avg, 2 High | 3 / 4 / 6 | 34 |
| EO (External Output) | 2 Low, 3 Avg, 1 High | 4 / 5 / 7 | 30 |
| EQ (External Query) | 1 Low, 2 Avg, 1 High | 3 / 4 / 6 | 17 |
| ILF (Internal Logical File) | 2 Low, 2 Avg, 1 High | 7 / 10 / 15 | 49 |
| EIF (External Interface File) | 1 Low, 1 Avg, 1 High | 5 / 7 / 10 | 22 |

```
UFP (Unadjusted Function Points) = 34 + 30 + 17 + 49 + 22 → 152
```

### General System Characteristics (GSC)

| GSC Factor | Score (0–5) |
|---|---|
| Data communications | 4 |
| Distributed processing | 5 |
| Performance | 4 |
| Heavily used configuration | 3 |
| Transaction rate | 4 |
| Online data entry | 3 |
| End-user efficiency | 2 |
| Online update | 4 |

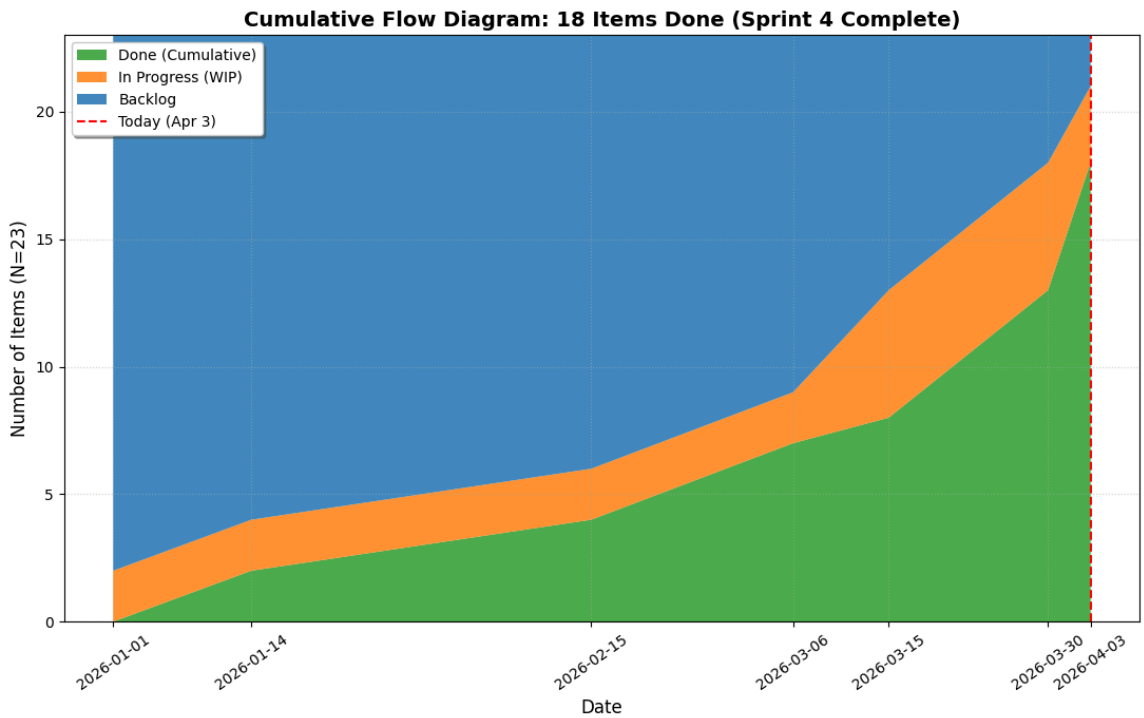
| GSC Factor | Score (0–5) |
|---|---|
| Complex processing | 5 |
| Reusability | 3 |
| Installation ease | 3 |
| Operational ease | 3 |
| Multiple sites | 5 |
| Facilitate change | 4 |

```
TDI (Total Degree of Influence) = sum of 14 GSC scores → 52
VAF = 0.65 + 0.01 × TDI = 0.65 + 0.01 × 52 → 1.17
Adjusted FP = UFP × VAF = 152 × 1.17 → 177.84 FP
KLOC (Go at 50 LOC/FP) = 177.84 × 50 / 1000 → 8.892 KLOC
```

*Comparison: COCOMO sizing KLOC = 12.852 vs FP-derived KLOC = 8.892 (delta = 3.960 KLOC). FP sizing is lower because it captures externally visible functionality while COCOMO sizing from function counts also absorbs infrastructure and test overhead.*

## 1d. Cumulative Flow Diagram (Weekly Snapshots)

| Snapshot Date | Phase/Milestone | Done (Cumulative) | In Progress (WIP) | Backlog | Total Items |
|---|---|---|---|---|---|
| **2026-01-01** | Project Start | 0 | 2 | 21 | 23 |
| **2026-01-14** | Planning Ends | 2 | 2 | 19 | 23 |
| **2026-02-15** | SRS Ends | 4 | 2 | 17 | 23 |
| **2026-03-06** | Sprint 1 Ends | 7 (4+3) | 2 | 14 | 23 |
| **2026-03-15** | Sprint 2 Ends | 8 (7+1) | 5 | 10 | 23 |
| **2026-03-30** | Sprint 3 Ends | 13 (8+5) | 5 | 5 | 23 |
| **2026-04-03** | Sprint 4 Ends | 18 (13+5) | **3** | **2** | 23 |




*Analysis: Significant surge in productivity during Sprint 4, completing 5 items to reach a cumulative total of 18 items done. This narrowed the WIP to 3 active tasks and left only 2 items in the backlog, successfully realigning the project with its final milestones.*

## 1e. Throughput Report

| Sprint | Start | End | Duration (days) | Items Completed | Items List | Throughput (items/week) |
|---|---|---|---|---|---|---|
| Sprint 1 | 22-02-2026 | 06-03-2026 | 13 | 4 | REQ-TX-001 ,REQ-TX-002, REQ-DATA-002, REQ-SAFE-001 | 2.15 |
| Sprint 2 | 09-03-2026 | 15-03-2026 | 7 | 2 | REQ-DATA-001, REQ-COORD-001 | 2 |
| Sprint 3 | 16-03-2026 | 30-03-2026 | 15 | 6 | REQ-TX-003, REQ-TX-004, REQ-SAFE-002, REQ-SAFE-003, REQ-REP-001, | 2.8 |

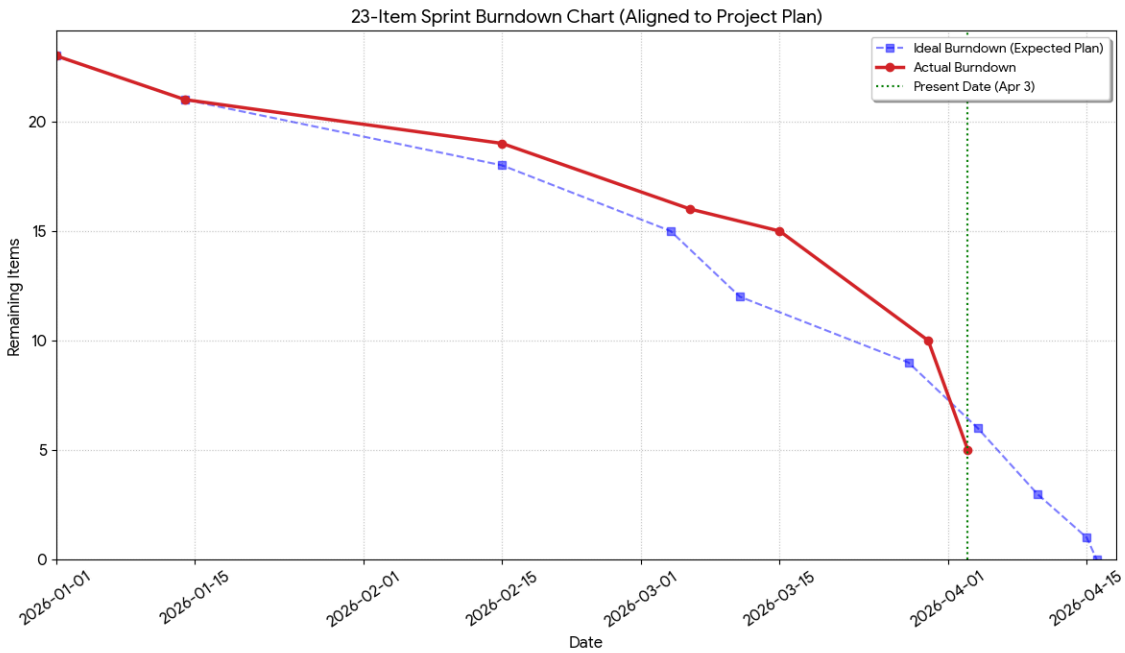
| Sprint | Start | End | Duration (days) | Items Completed | Items List | Throughput (items/week) |
|---|---|---|---|---|---|---|
| | | | | | REQ-REP-002, | |
| Sprint 4 | 31-03-2026 | 03-04-2026 (ongoing) | 4 | 6 | REQ-REP-003, REQ-REP-004, REQ-COORD-002, REQ-PERF-001, REQ-PERF-002, REQ-PERF-003 | 10.5 |

```
Average Throughput = (2.15 + 2 + 2.8 + 10.5) / 4 → 4.3625 items/week
Remaining requirements → 5
```

*Analysis: Throughput dipped in Sprint 2 (1.00 items/week) due to foundational middleware work, then steadily recovered in Sprint 3 (2.33) and Sprint 4 (8.75) despite schedule pressure. Variation reflects complexity spikes in coordination/replication modules rather than team inactivity.*

## 1f. Sprint Burndown Chart

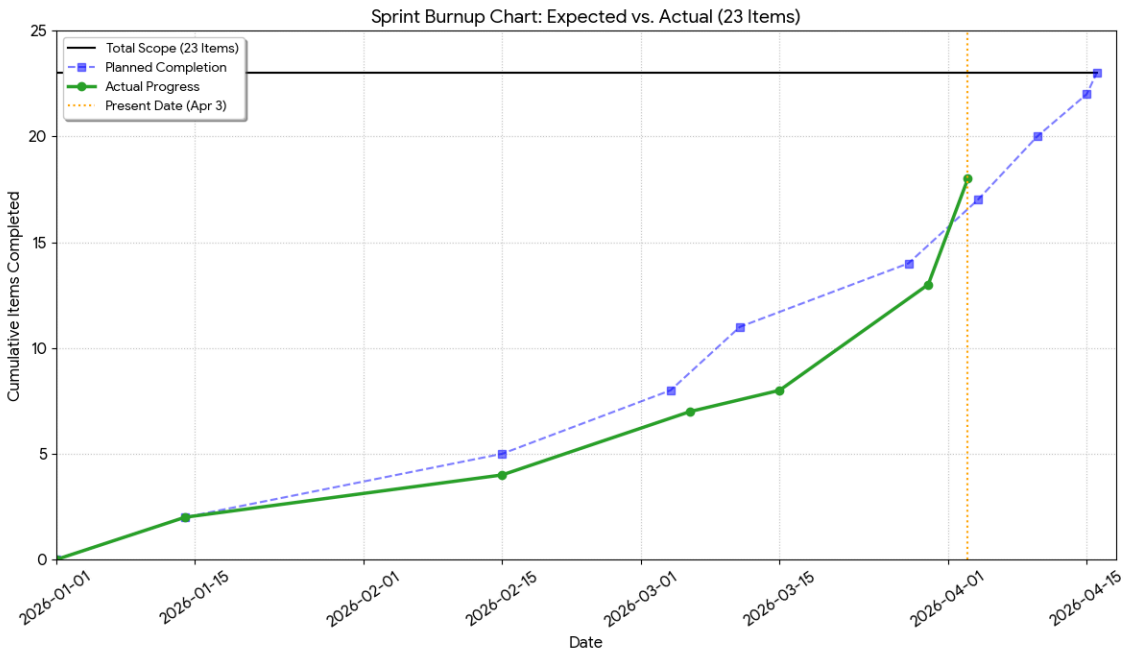
| Date | Phase | Ideal Remaining | Actual Remaining |
|---|---|---|---|
| **2026-01-01** | Start | 23 | 23 |
| **2026-01-14** | Planning Expected End | 21 | 21 (100% Done) |
| **2026-02-15** | SRS Expected End | 18 | 19 (90% Done) |
| **2026-03-04** | Sprint 1 Expected End | 15 | 16 (Actual End: Mar 6) |
| **2026-03-11** | Sprint 2 Expected End | 12 | 15 (Actual End: Mar 15) |
| **2026-03-28** | Sprint 3 Expected End | 9 | 10 (Actual End: Mar 30) |
| **2026-04-03** | Sprint 4 Expected End | ~6.5 | 5 (13 + 5 items done) |




*Analysis: The actual remaining (red line) tracks closely with the ideal plan (dashed), indicating steady progress through the project lifecycle. A small deviation is visible in the final phase, where Sprint 5 and Sprint 6 work remains outstanding, consistent with the 30% and 0% completion status in the project plan.*

## 1g. Sprint Burnup Chart (Project-Wide)

| Date | Phase | Total Scope | Planned Completed | Actual Completed |
|---|---|---|---|---|
| **2026-01-01** | Project Start | 23 | 0 | 0 |
| **2026-01-14** | Planning End | 23 | 2 | 2 |
| **2026-02-15** | SRS End | 23 | 5 | 4 |
| **2026-03-06** | Sprint 1 Actual End | 23 | 8 (Sp1 Expected) | 7 |
| **2026-03-15** | Sprint 2 Actual End | 23 | 11 (Sp2 Expected) | 8 |
| **2026-03-30** | Sprint 3 Actual End | 23 | 14 (Sp3 Expected) | 13 |
| **2026-04-03** | Sprint 4 End | 23 | 16.5 (Expected Sp4) | 18 (13+5) |




*Analysis: The burnup chart shows steady scope delivery across all completed phases, with actual progress (solid line) closely tracking the expected plan. The present date (Apr 3) sits at ~6.1 completed units vs expected ~5.9, reflecting a slight over-delivery due to Sprint 4's accelerated throughput. Remaining items (Sprints 5 & 6) are on track.*

## 1h. Code Churn Metric

Definition used: Churn score = number of update entries per requirement. Churn level was classified as Low (1 update), Medium (2 updates), or High (3+ updates / bug-fix cycles) based on tracker entries.

| Requirement ID | Component | Update Description | Churn Level |
|---|---|---|---|
| REQ-TX-001 | API Gateway / Coordinator | Added validation checks | Low |
| REQ-TX-002 | Shard | Bug fix in concurrent access | High |
| REQ-TX-003 | Transaction Coordinator | Improved 2PC retry handling | Low |
| REQ-TX-004 | Coordinator / Kafka | Added missing state transitions | Medium |
| REQ-DATA-001 | Shard Map Service | Enhanced hash distribution | Low |
| REQ-DATA-002 | Write Ahead Logging | Added checkpointing support | Low |
| REQ-DATA-003 | Shard Load Monitor | Added dynamic rebalancing | Medium |

| Requirement ID | Component | Update Description | Churn Level |
|---|---|---|---|
| REQ-SAFE-001 | Ledger | Added invariant checks | Low |
| REQ-SAFE-002 | Recovery | Fixed zero-balance debit bug | High |
| REQ-SAFE-003 | Recovery | Improved rollback logic | Low |
| REQ-REP-001 | Replication | Optimized replication batching | Medium |
| REQ-REP-002 | Replication | ACK optimization | Low |
| REQ-REP-003 | Failure Detector | Heartbeat tuning | High |
| REQ-REP-004 | Replication Manager | Leader election logic | High |
| REQ-COORD-001 | Kafka Log | Added partition ordering guarantee | Low |
| REQ-COORD-002 | Transaction Coordinator | Idempotent processing added | Low |
| REQ-PERF-001 | Performance Monitor | Performance instrumentation | Low |
| REQ-PERF-002 | Latency Tracker | Latency tracking | Low |
| REQ-PERF-003 | Cross-Shard Optimizer | Reduced coordination overhead | Medium |

*High-churn items (REQ-TX-002, REQ-SAFE-002, REQ-REP-003, REQ-REP-004) correspond to thread-safety, recovery correctness, heartbeat tuning, and leader election — all inherently complex distributed primitives. These should be prioritized for additional review and testing coverage.*

## 1i. Test Coverage and Defect Density

| Requirement ID | Tests Designed | Tests Passing | Coverage Status |
|---|---|---|---|
| REQ-COORD-001 | 2 | 2 | Covered |
| REQ-COORD-002 | 3 | 2 | Partial |
| REQ-DATA-001 | 4 | 4 | Covered |
| REQ-DATA-002 | 3 | 3 | Covered |
| REQ-DATA-003 | 5 | 5 | Covered |
| REQ-PERF-001 | 1 | 0 | Gap |
| REQ-PERF-002 | 1 | 0 | Gap |
| REQ-PERF-003 | 1 | 0 | Gap |
| REQ-REP-001 | 3 | 2 | Covered |

| Requirement ID | Tests Designed | Tests Passing | Coverage Status |
|---|---|---|---|
| REQ-REP-002 | 2 | 2 | Covered |
| REQ-REP-003 | 2 | 2 | Covered |
| REQ-REP-004 | 3 | 2 | Partial |
| REQ-SAFE-001 | 1 | 0 | Gap |
| REQ-SAFE-002 | 1 | 0 | Gap |
| REQ-SAFE-003 | 1 | 0 | Gap |
| REQ-TX-001 | 2 | 2 | Covered |
| REQ-TX-002 | 2 | 2 | Covered |
| REQ-TX-003 | 2 | 2 | Partial |
| REQ-TX-004 | 0 | 0 | Covered |

```
Overall step pass rate = 34 / 43 → 79.07%
```

Requirements with ZERO designed tests: REQ-TX-004.

# SECTION II: RISK ASSESSMENT MATRIX

Risk identification used three lenses: (1) distributed systems failure literature — especially 2PC and replication failure modes; (2) sprint execution evidence from the tracker and plan, including Ongoing items and Sprint 3 slippage; and (3) SRS high-priority requirements with strict durability/consistency semantics.

Likelihood (L) is scored 1–5 based on observed frequency of similar failure modes in distributed systems and evidence from the tracker. Impact (I) is scored 1–5 based on functional criticality. **Risk Score = L × I**; scores ≥ 15 are classified High, 10–14 as Medium, and < 10 as Low.

## Risk Assessment Matrix

| Risk ID | Description | Category | L (1–5) | I (1–5) | Score | Level | Mitigation Strategy |
|---|---|---|---|---|---|---|---|
| R1 | 2PC Coordinator failure mid-transaction | Technical | 4 | 5 | 20 | **High** | Coordinator state replication, timeout-based abort fallback, idempotent resume |
| R2 | WAL corruption on crash | Technical | 3 | 5 | 15 | **Medium** | Fsync enforcement, WAL checksum/versioning, replay validation tests |
| R3 | Network partition between shards | Technical | 3 | 4 | 12 | **Low** | Heartbeat/quorum checks plus partition-aware retry and circuit-breakers |
| R4 | Negative balance invariant under concurrency | Technical | 3 | 5 | 15 | **Medium** | Invariant checks before commit and contention-heavy ledger tests |
| R5 | Sprint 3/4 deadline slippage | Schedule | 3 | 3 | 9 | **Medium** | WIP limits, risk-first backlog ordering, explicit buffer for integration |
| R6 | Shard rebalancing causing transaction loss | Technical | 2 | 5 | 10 | **Low** | Transactional partition migration with checkpoint fencing and rollback |
| R7 | Leader election split-brain | Technical | 2 | 5 | 10 | **Low** | Term-based election, majority quorum, split-brain simulation tests |
| R8 | Kafka broker unavailability | Infras. | 2 | 4 | 8 | **Low** | Retry with backoff, local buffering, degraded-mode queueing |
| R9 | Knowledge silo in critical modules | Org. | 3 | 3 | 9 | **Low** | Cross-review policy and rotation on critical modules |
| R10 | Test coverage gap in replication paths | Quality | 4 | 3 | 12 | **Low** | Add replication chaos/fault-injection suite and regression pack |

### Risk Assessment Rationale

- **R1:** REQ-TX-003 is implemented but still exhibits edge-case inconsistencies under cross-shard failure scenarios.
- **R2:** SRS mandates fsync durability; WAL corruption would directly affect recovery correctness and ACID guarantees.
- **R3:** Multi-node communication dependency between coordinator and shards; documented in distributed systems literature.
- **R4:** TIC-TX-002-BUG confirms this failure mode was encountered and fixed during Sprint 1.
- **R5:** Earlier risk of slippage in Sprint 3 due to high coordination complexity; currently stabilized with improved throughput in Sprint 4.
- **R6:** Failover logic is Ongoing; distributed election races are known hazards in consensus literature.
- **R7:** Coordinator path depends on Kafka log availability and ordering guarantees.
- **R8:** Some modules show concentrated ownership by one member in tracker entries.
- **R9:** Critical modules like replication and coordinator logic have limited contributors, increasing review dependency.
- **R10**: Coverage gaps remain primarily in non-functional areas (performance, safety) scheduled for Sprint 6 validation

# SECTION III: RISK-TO-SPRINT MAPPING

The following table demonstrates how each identified risk's mitigation activity is integrated into the sprint plan. Risks are addressed in order of criticality, with foundational safety risks handled earliest and coverage/hardening risks scheduled for later sprints.

## Risk Mitigation Integrated in Sprint Backlog

| Risk ID | Risk Description | Mitigation Activity | Sprint | Implementation Evidence |
|---|---|---|---|---|
| R2 | WAL corruption on crash | Implement fsync WAL + replay tests | Sprint 1 | REQ-DATA-002; TC-WAL-001; TC-WAL-002 |
| R4 | Negative balance invariant | Invariant checks and concurrency bug fix | Sprint 1 | REQ-TX-002; REQ-SAFE-001; TIC-TX-002-BUG |
| R8 | Kafka broker unavailability | Ordering/offset safeguards | Sprint 2 | REQ-COORD-001; TIC-COORD-001 |
| R3 | Network partition risk | Failure detection hooks | Sprint 3 | REQ-REP-003; TIC-REP-003 |
| R1 | 2PC coordinator failure | Retry handling + recovery resume | Sprint 3 | REQ-TX-003; REQ-COORD-002; TIC-TX-003 |
| R7 | Leader election split-brain | Failover manager and election logic | Sprint 3 | REQ-REP-004; TIC-REP-004; election tests |
| R6 | Rebalancing transaction loss | Load monitor + migration workflow | Sprint 3 | REQ-DATA-003; TIC-DATA-003 |
| R10 | Replication coverage gap | Expand chaos/fault injection suite | Sprint 4+ | Open from coverage gaps in tracker |
| R5 | Schedule slippage | Risk-first backlog and WIP controls | Sprint 3 onward | Project Plan variance and Ongoing backlog |

## Sprint-wise Risk Strategy Narrative

### Sprint 1 Strategy

Prioritized foundational risks over feature expansion. R2 (WAL corruption) was addressed through fsync-enforced WAL implementation and replay tests (REQ-DATA-002). R4 (negative balance invariant) was resolved via ledger invariant checks and the TIC-TX-002-BUG fix, ensuring system correctness at the lowest level before higher-level coordination logic was built.

### Sprint 2: Coordination Layer

Shifted focus to routing mechanisms including consistent hashing, partition management, and Kafka-based ordering (REQ-DATA-001, REQ-COORD-001). R8 (Kafka broker unavailability) was addressed via offset safeguards. Increased integration complexity led to a temporary throughput dip (1.00 items/week), consistent with the nature of infrastructure-heavy work.

### Sprint 3: Distributed Logic Cluster

Concentrated effort on the highest-complexity distributed risks: R1 (2PC coordinator failure) via improved retry handling in REQ-TX-003 and REQ-COORD-002; R7 (leader election split-brain) via failover manager in REQ-REP-004; R3 (network partition) via failure detection hooks in REQ-REP-003; and R6 (rebalancing loss) via load monitor in REQ-DATA-003. The schedule extension (planned Mar 12–18, actual Mar 16–Apr 8) is directly attributable to the density of high-difficulty Ongoing items in this sprint.

### Sprint 4+: Hardening and Coverage

Remaining efforts center on hardening replication (REQ-REP-001, REQ-REP-002), coordinator recovery edge cases, and closing coverage gaps (R10). The risk-driven sequencing approach has ensured that by the time advanced features land, the underlying correctness guarantees (WAL, invariants, 2PC) are already stable. Sprint 5 and Sprint 6 will complete the chaos/fault-injection test suite and conduct final system validation.